Polytech Dijon

Mécatronique

Travaux Pratique

Compte rendu

Réalisé par :
BAKAI Mamalineso
CLOIX Mattys

Supervisé par :
ABADI Amine

13 octobre 2026

Table des matières

1 TP1 :Présentation et programmation de la carte Arduino 2
- 1.1 Matériel . . . 3
- 1.2 Le logiciel Arduino . . . 4
- 1.3 Manipulation . . . 4
  - 1.3.1 Manipulation 1 : Une LED qui clignote . . . 4
  - 1.3.2 Manipulation 2 : Trois LED qui clignote . . . 5

2 TP2 :Utilisation du CAN de l'Arduino 11
- 2.1 Convertisseur Analogique numérique . . . 12
- 2.2 Mesurer une température avec un capteur LM35 et une carte Arduino . . . 12
- 2.3 Mesurer la luminosité avec une photorésistance et une carte Arduino . . . 14
- 2.4 Variation de la luminosité d'une LED avec un potentiomètre et une carte Arduino . . . 17

3 TP 3 : Afficheurs 7 segments, MATLAB/Simulink et Asservissement 21
- 3.1 Utilisation des afficheurs 7 segments . . . 22
- 3.2 Interfaçage Arduino et MATLAB/Simulink . . . 25
- 3.3 Asservissement en vitesse d'un moteur avec régulateur PID . . . 27

4 TP : Commande des moteurs avec Arduino 29
- 4.1 Moteur à courant continu et module relais . . . 30
- 4.2 Signal PWM et servomoteur . . . 32

Chapitre 1

TP1 :Présentation et programmation de la carte Arduino

1.1 Matériel

Le projet Arduino a été conçu par une équipe de six développeurs pour offrir une plateforme de création de systèmes électroniques accessible à tous, des débutants aux professionnels.

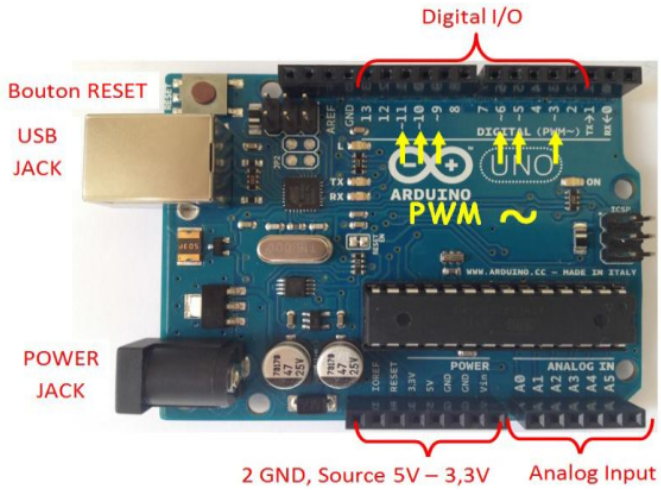


FIGURE 1.1 – Arduino Uno

La carte Arduino UNO est un circuit imprimé architecturé autour du microcontrôleur ATmega328. Elle est équipée d'une interface USB permettant une communication directe avec un ordinateur.

Caractéristiques techniques principales :

- **Tensions :** Fonctionnement sous 5V, avec une tension d'alimentation recommandée de 7 à 12V.
- **Performances :** Horloge cadencée à 16 MHz et mémoire Flash de 32 Ko.
- **Courant :** Jusqu'à 40 mA par broche d'entrée/sortie (50 mA pour la broche 3.3V).
- **Brochage :**
  - *Entrées analogiques :* A0 à A5
  - *Entrées/Sorties numériques :* 2 à 19
  - *Sorties PWM (analogiques simulées) :* 3, 5, 6, 9, 10, 11

1.2 Le logiciel Arduino

La programmation de la carte Arduino nécessite un logiciel pour écrire le code puis le transférer. Nous utiliserons pour cela Arduino IDE, une solution gratuite et open source.

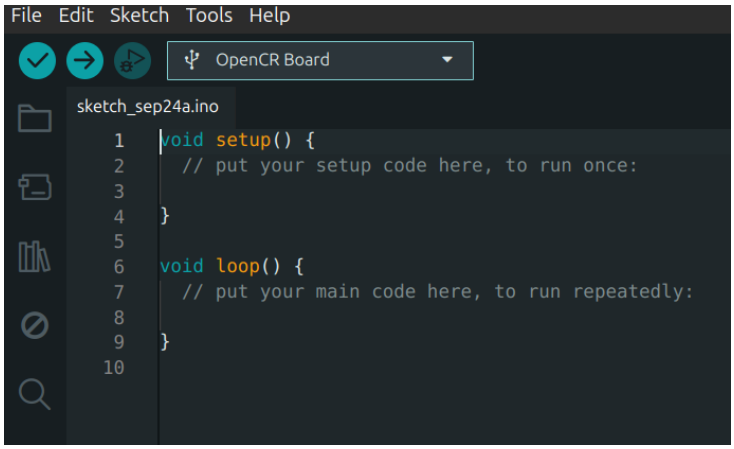


FIGURE 1.2 – Interface Arduino IDE

1.3 Manipulation

1.3.1 Manipulation 1 : Une LED qui clignote

Le but de cette première manipulation est de nous familiariser avec la carte Arduino Uno en faisant simplement clignoter une LED.

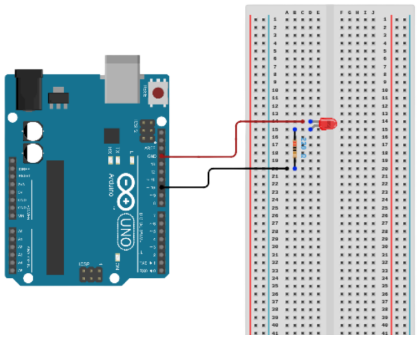


FIGURE 1.3 – Schéma Manipulation 1

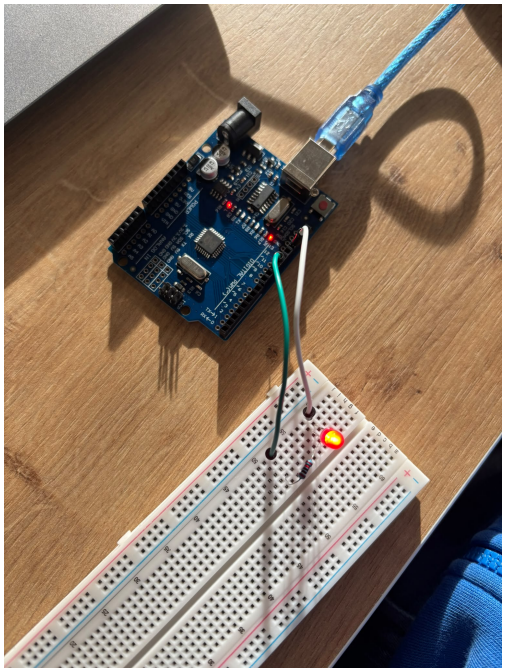


FIGURE 1.4 – Photo Manipulation 1

Voici le code qui nous est donné pour tester notre LED

```
1  void setup() {
2      pinMode(10, OUTPUT); // ici nous definissons notre
           pin 10 comme une sortie
3  }
4  void loop() {
5      digitalWrite(10, HIGH); // ici on allume la LED
6      delay(500); // on attends 500 millisecondes
7      digitalWrite(10, LOW);
8      delay(500); // on attends 500 millisecondes
9  }
```

Observation : Nous remarquons bien que la LED clignote.

1.3.2 Manipulation 2 : Trois LED qui clignote

Dans cette partie du TP, nous devons apprendre à gérer trois LED en même temps avec notre Arduino.

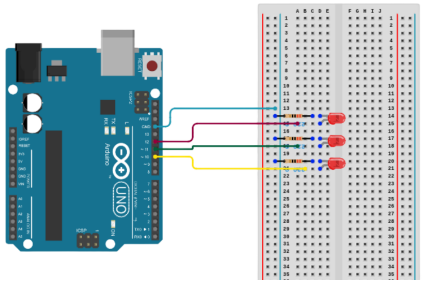


FIGURE 1.5 – Schéma manipulation 2

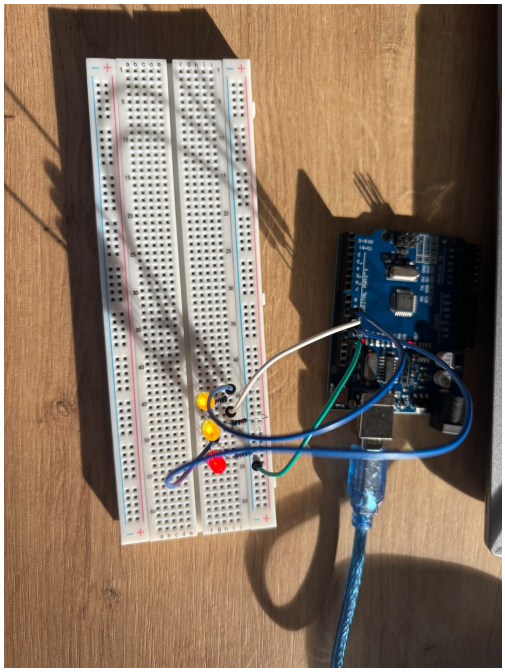


FIGURE 1.6 – Photo Manipulation 2

Voici notre code pour faire code pour faire clignoter les trois LED

```
1  void setup() {
2      pinMode(10, OUTPUT); // ici nous definissons notre
           pin 10 comme une sortie
3      pinMode(11, OUTPUT); // ici nous definissons notre
           pin 11 comme une sortie
4      pinMode(12, OUTPUT); // ici nous definissons notre
           pin 12 comme une sortie
5  }
6  void loop() {
7      digitalWrite(10, HIGH); // ici on allume la LED
8      digitalWrite(11, HIGH); // ici on allume la LED
9      digitalWrite(12, HIGH); // ici on allume la LED
10
11     delay(500); // on attends 500 millisecondes
12
13     digitalWrite(10, LOW);
14     digitalWrite(11, LOW);
15     digitalWrite(12, LOW);
16
17     delay(500); // on attends 500 millisecondes
18 }
```

Observation : Nous remarquons que les trois LED s'allume simultanément.

Ensuite nous devons faire en sorte que les trois Led s'allume avec alternance. Voici le code pour faire cela.

```
    void loop() {
        digitalWrite(10, HIGH); // ici on allume la LED
        delay(500); // on attends 500 millisecondes
        digitalWrite(10, LOW); // ici on allume la LED

        delay(500);

        digitalWrite(11, HIGH); // ici on allume la LED
        delay(500); // on attends 500 millisecondes
        digitalWrite(11, LOW); // ici on allume la LED

        delay(500);

        digitalWrite(12, HIGH); // ici on allume la LED
        delay(500); // on attends 500 millisecondes
        digitalWrite(12, LOW); // ici on allume la LED

        delay(500);
    }
```

Observations : Nous remarquons bien une alternance dans l'allumage des LED.
Pour la prochaine questions nous devons réaliser un feu tricolore avec nos trois LED de couleur. Voici alors notre code pour réaliser cela.

```
    #define LED_rouge  12 // ici nous renommons nous leds
       pour mieux les distinguer
    #define LED_vert   11
    #define LED_orange 10
    void setup() {
        pinMode(LED_rouge, OUTPUT);
        pinMode(LED_vert, OUTPUT);
        pinMode(LED_orange, OUTPUT);
    }
    void loop() {

        digitalWrite(LED_orange, HIGH);
        digitalWrite(LED_vert, LOW);
        digitalWrite(LED_rouge, LOW);

        delay(1000);

        digitalWrite(LED_orange, LOW);
        digitalWrite(LED_vert, LOW);
        digitalWrite(LED_rouge, HIGH);

```

```
        delay(4000);

        digitalWrite(LED_orange, LOW);
        digitalWrite(LED_vert, HIGH);
        digitalWrite(LED_rouge, LOW);

        delay(4000);
    }
```

Observations : Nous remarquons bien le fait que les LED s'allume a l'image d'un feux tricolore.
L'exercice suivant consiste à répéter 15 fois le clignotement d'une seule LED. Pour éviter d'alourdir le programme, nous allons conserver notre structure actuelle et introduire une boucle for.

```
    void setup() {
        Serial.begin(9600);

        pinMode(10, OUTPUT);

        for(int i=0 ; i <15 ; i++){ // cette boucle for nous
            permet d'executer le code 15 fois
            Serial.println(i);
            digitalWrite(10,HIGH);
            delay(500);
            digitalWrite(10,LOW);
            delay(500);
        }
    }
    void loop() {
        // ici on ne mets rien car on souhaite que le code
            sexecute que 15 fois
    }
```

Observations : Nous vérifions bien que la LED s'allume et s'éteint 15 fois. Nous avons désormais un code a tester.

```
    int ledPin = 10; //On renomme la broche 10 en "ledPin"
    int timer = 100; //On definit une duree de 0,1 seconde
       pour la variable timer

    void setup() { pinMode(ledPin, OUTPUT); }

    void loop() {
        // LED a 0%.
        analogWrite(ledPin, 0);
        delay(timer);
        // LED a 19.6%.
        analogWrite(ledPin, 50);
```

```
        delay(timer);
        // LED a 39.2%.
        analogWrite(ledPin, 100);
        delay(timer);
        // LED a 58.8%.
        analogWrite(ledPin, 150);
        delay(timer);
        // LED a 78.4%.
        analogWrite(ledPin, 200);
        delay(timer);
        // LED a 100%.
        analogWrite(ledPin, 255);
        delay(timer); }
```

Observations : La LED augmente en luminosité jusqu'à s'éteindre.
Maintenant que nous avons compris ce précedent code nous devons jouer avec la luminosité des Trois différente LED.

```
int ledPin = 10; //On renomme la broche 10 en "ledPin"
int timer = 100; //On definit une duree de 0,1 seconde
    pour la variable timer

void setup() {
    pinMode(ledPin, OUTPUT);
}

void loop() {
    analogWrite(10, (255*10)/100); // ici on definit la
        luminosite de la led_10 a 10% de 255
    analogWrite(11, (255*40)/100); // ici on definit la
        luminosite de la led_11 a 40% de 255
    analogWrite(12, (255*80)/100); // ici on definit la
        luminosite de la led_12 a 80% de 255

    delay(timer);

    analogWrite(10, (255*0)/100); // ici on definit la
        luminosite de la led_10 a 0% de 255
    analogWrite(11, (255*0)/100); // ici on definit la
        luminosite de la led_11 a 0% de 255
    analogWrite(12, (255*0)/100); // ici on definit la
        luminosite de la led_12 a 0% de 255

    delay(timer);
}
```

Observations : Les trois LED clignote mais avec une intensité lumineuse différente.
Pour finir avec ce TP nous allons réaliser un chenillard avec nos trois LED.

```
1   int timer = 500; //On definit une duree de 0,5 seconde
        pour la variable timer
2   void setup() {
3       pinMode(10, OUTPUT);
4       pinMode(11, OUTPUT);
5       pinMode(12, OUTPUT);
6   }
7   void loop() {
8       for (int i = 10 ; i < 13 ; i++) {
9           digitalWrite(i, HIGH);
10          delay(timer);
11          digitalWrite(i, LOW);
12      }
13  }
```

Observations : Nous observons bien l'allumage des LED de façons successive.

Chapitre 2

TP2 :Utilisation du CAN de l'Arduino

2.1 Convertisseur Analogique numérique

La carte Arduino Uno intègre un unique convertisseur analogique/numérique (CAN) de 10 bits, partagé entre ses 6 entrées (A0 à A5). Il convertit le signal en environ 100 µs pour délivrer une valeur numérique allant de 0 à 1023.

2.2 Mesurer une température avec un capteur LM35 et une carte Arduino

Le LM35 est un capteur de température électronique analogique de précision. Il se présente sous la forme d'un composant à trois broches :

— **VCC** : Alimentation (reliée au 5V)

— **Analog Voltage Out** : Sortie de la tension (reliée à une entrée analogique)

— **GND** : Masse

Principe de mesure : D'après la fiche technique du composant, la tension de sortie est proportionnelle à la température avec une sensibilité de 10 mV/°C. Autrement dit, une tension de 1 V correspond à 100°C.

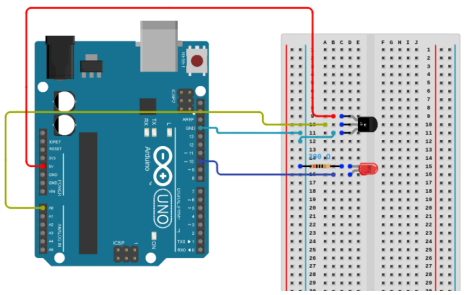


FIGURE 2.1 – Schéma de l'utilisation du LM35

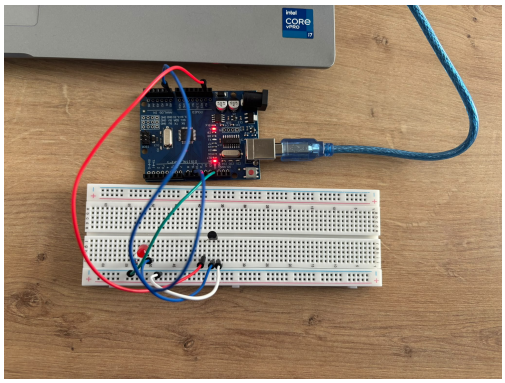


FIGURE 2.2 – Photo du montage du LM35

Pour commencer nous allons programmer la carte pour uniquement récupérer les informations du LM35.

```arduino
const int lm35Pin = A0; // Broche du capteur LM35

void setup() {
    Serial.begin(9600); // Initialisation de la communication serie
}
void loop() {
    int val = analogRead(lm35Pin); // Lecture de la valeur analogique (0 a 1023)

    // Conversion de la valeur lue en tension (en mV), puis en temperature (C)

    float voltage = val * (5.0 / 1023.0);
    float temperature = voltage * 100.0; // 10 mV par C donc T = tension * 100

    Serial.print("Temperature : ");
    Serial.print(temperature);
    Serial.println(" C");

    delay(1000); // Pause d'une seconde entre chaque mesure
}
```

Observations : Nous utilisons le moniteur série de l'Arduino pour relever la température mesurée par le capteur. La valeur obtenue étant d'environ 22 °C, nous pouvons en déduire que le montage et la programmation ont été réalisés avec succès.

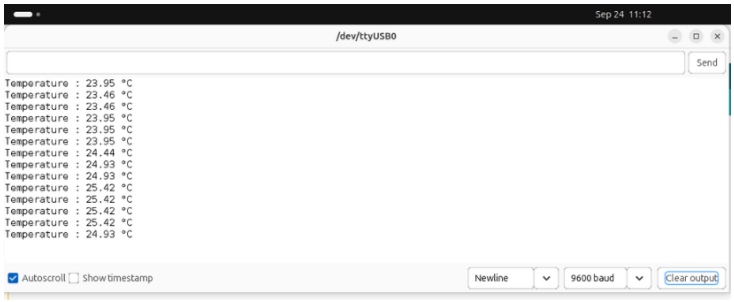


FIGURE 2.3 – Monitor série du capteur LM35

Nous faisons évoluer le programme pour utiliser la LED comme indicateur d'alerte. Si la température mesurée par le LM35 est supérieure à 35 °C, nous commandons l'allumage de la LED.

```arduino
const int lm35Pin = A0; // Broche du capteur LM35
void setup() {
```

```
    Serial.begin(9600); // Initialisation de la communication serie
    pinMode(10, OUTPUT);
}
void loop() {
    int val = analogRead(lm35Pin); // Lecture de la valeur analogique (0 a 1023)

    // Conversion de la valeur lue en tension (en mV), puis en temperature (C)

    float voltage = val * (5.0 / 1023.0);
    float temperature = voltage * 100.0; // 10 mV par C donc T = tension * 100

    Serial.print("Temperature : ");
    Serial.print(temperature);
    Serial.println(" C");

    delay(1000); // Pause d'une seconde entre chaque mesure

    if (temperature > 29) {
        digitalWrite(10,HIGH); // on allume la LED
    }else {
        digitalWrite(10,LOW); // on eteint la LED
    }
}
```

Observations : Lorsque que nous plaçons notre doigts sur le LM35 alors la LED s'allume car nous dépassons le seuil de température de notre programme.

2.3 Mesurer la luminosité avec une photorésistance et une carte Arduino

Une **photorésistance** est un composant électronique dont la valeur (en ohms) varie en fonction de la lumière ambiante : **plus elle est éclairée, plus sa résistance diminue**.

Caractéristiques classiques

- **Dans l'obscurité :** Résistance très élevée (environ 1 MΩ).
- **En pleine lumière :** Résistance plus faible (environ 12 kΩ).

Applications principales Ce comportement permet de mesurer la luminosité ambiante pour diverses utilisations :

— Détection de l'état jour/nuit.

— Ajustement automatique de l'éclairage.

— Systèmes de suivi de lumière (panneaux solaires, robotique).

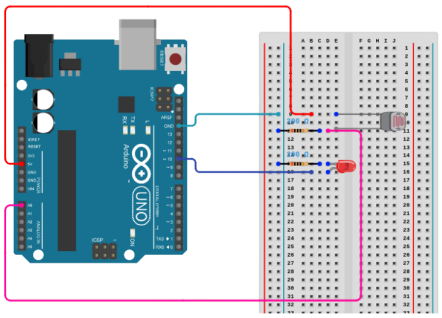


FIGURE 2.4 – Schéma du montage avec photorésistance

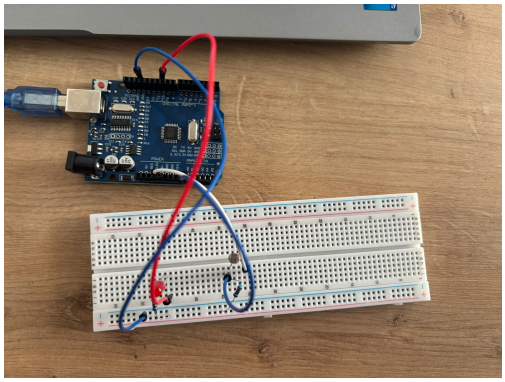


FIGURE 2.5 – Photo montage avec la photorésistance

Ensuite, nous allons faire un programme permettant de gérer l'allumage d'une LED en fonction de la luminosité reçue par la photorésistance.

```
1   // Initialisation des constantes :
2   const int analogInPin = A0; // Numero de la broche a
        laquelle est connectee la photoresistance
3   const int analogOutPin = 10; // Numero de la broche a
        laquelle est connectee la LED
4
5   int sensorValue = 0; // Valeur lue sur la photoresistance
6   int outputValue = 0; // Valeur envoyee a la LED
7   void setup() {
8       // Initialise la communication avec l'ordinateur
9       Serial.begin(9600);
10      // Indique que la broche analogOutPin est une sortie
            :
11      pinMode(analogOutPin, OUTPUT);
12      // Indique que la broche analogInPin est une entree :
13      pinMode(analogInPin, INPUT);
14  }
15  void loop() {
16      // lit la valeur de la photoresistance et
17      // stocke le resultat dans sensorValue :
18      sensorValue = analogRead(analogInPin);
```

```
        // change sensorValue vers une intervalle de 0 a 255
        // et stocke le resultat dans outputValue :
        outputValue = map(sensorValue, 0, 1023, 0, 255);
        // envoie de cette nouvelle valeur sur la LED
        analogWrite(analogOutPin, outputValue);
        Serial.print("sensor = " );
        Serial.print(sensorValue);
        Serial.print("\t output = ");
        Serial.println(outputValue);
    }
```

Observations : Lorsque nous mettons notre doigt sur la photorésistance pour baiser la luminosité alors la LED s'allume. Nous allons rajouter une autre LED qui s'allume avec un autre seuil.

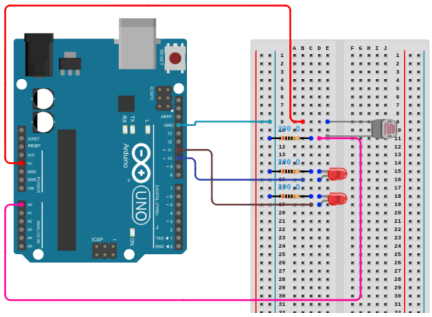


FIGURE 2.6 – Schéma du montage avec photorésistance et deux LED

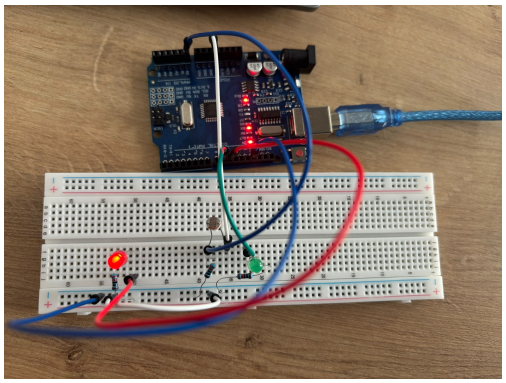


FIGURE 2.7 – Photo montage avec la photorésistance et deux LED

```
void loop() {
    sensorValue = analogRead(analogInPin);

    outputValue = map(sensorValue, 0, 1023, 0, 255);

    analogWrite(analogOutPin, outputValue);

    Serial.print("sensor = " );
    Serial.print(sensorValue);
    Serial.print("\t output = ");
    Serial.println(outputValue);
    if(sensorValue < ((1023*1)/100) ){
        digitalWrite(11, HIGH);
    }else{
        digitalWrite(11, HIGH);
    }
}
```

Observations : Les deux LED s'allume bien lorsque en fonction du seuil de luminosité atteint.

2.4 Variation de la luminosité d'une LED avec un potentiomètre et une carte Arduino

Pour terminer cette manipulation, nous intégrons un potentiomètre au lieu d'un capteur automatique. Cela nous donne la possibilité de contrôler manuellement le circuit en modifiant la valeur de sa résistance.

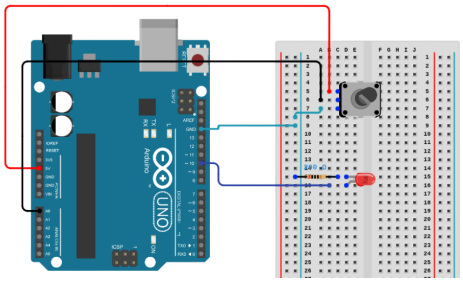


FIGURE 2.8 – Schéma du montage avec le potentiomètre

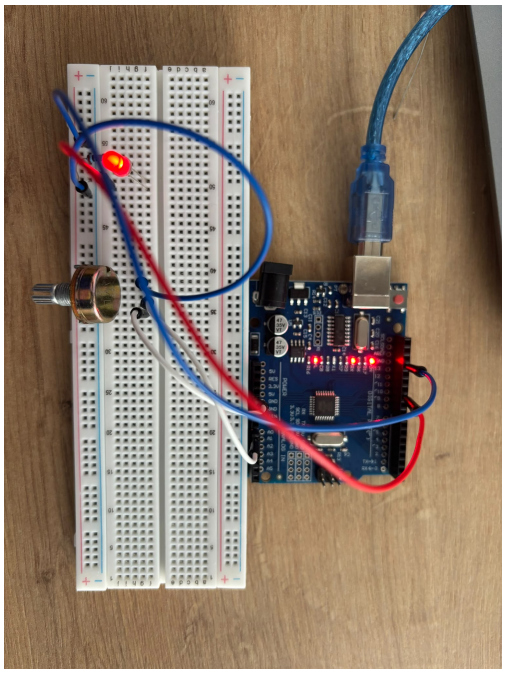


FIGURE 2.9 – Photo montage avec le potentiomètre

Nous modifions alors un peu le code de la photorésistance pour utiliser le potentiomètre.

```
1   void loop() {
2
3       sensorValue = analogRead(analogInPin);
4
5       outputValue = map(sensorValue, 0, 1023, 0, 255);
6
7       analogWrite(analogOutPin, outputValue);
8
```

```
9           Serial.print("sensor = " );
10          Serial.print(sensorValue);
11          Serial.print("\t output = ");
12          Serial.println(outputValue);
13      }
```

Observations : La LED s'allume bien lorsque que nous tournons le potentiomètre.
Nous ajoutons ensuite une autre LED qui devras s'allumer quand le potentiomètre est au minimum.

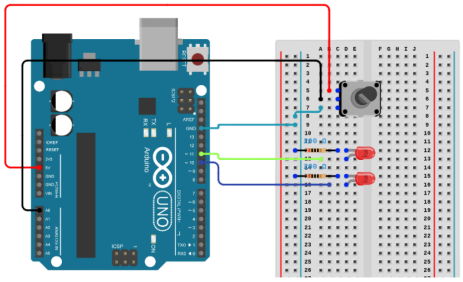


FIGURE 2.10 – Schéma du montage du potentiométre avec 2 LED

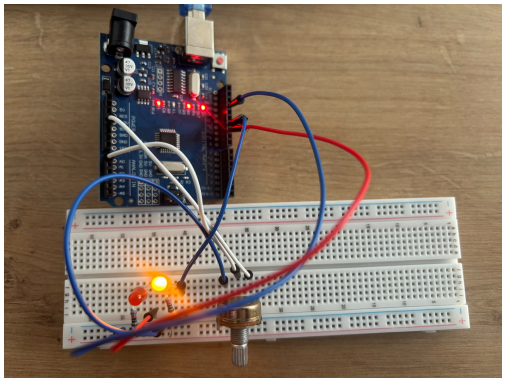


FIGURE 2.11 – Photo montage du potentiométre avec 2 LED

```
1       void loop() {
2
3           sensorValue = analogRead(analogInPin);
4
5           outputValue = map(sensorValue, 0, 1023, 0, 255);
6           outputValue2 = map(sensorValue, 0, 1023, 255, 0); //
                on inverse le 255
7
8           analogWrite(analogOutPin, outputValue);
9           analogWrite(11, outputValue2);
10
11          Serial.print("sensor = " );
12          Serial.print(sensorValue);
13          Serial.print("\t output = ");
14          Serial.println(outputValue);
15
16      }
```

Observations : Tourner le potentiomètre dans un sens allume une première LED, le tourner dans l'autre sens allume la seconde.

Pour finir avec ce TP nous devons afficher sur un écran LCD la température que nous mesurons a l'aide du LM35 utilisé dans la première partie du TP. Pour ce faire nous utilisons un code qui a été fournie par une bibliothèque trouvé sur Google.

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
const int lm35Pin = A0; // Broche du capteur LM35
// Initialisation du LCD I2C : Adresse 0x27, 16 colonnes, 2 lignes
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
    // Initialisation du LCD
    lcd.init();
    lcd.backlight(); // Allume le retroeclairage
    lcd.clear();
}
void loop() {
    int val = analogRead(lm35Pin); // Lecture du capteur

    // Conversion en temperature (C)
    float voltage = val * (5.0 / 1023.0);
    float temperature = voltage * 100.0;

    // Affichage sur l'ecran LCD
    lcd.setCursor(0, 0); // Ligne 1
    lcd.print("Temp: ");
    lcd.print(temperature, 1); // 1 chiffre apres la virgule
    lcd.print(" C   "); // Espaces pour eviter les bugs d'affichage

    delay(1000); // Pause d'une seconde avant la prochaine mesure
}
```

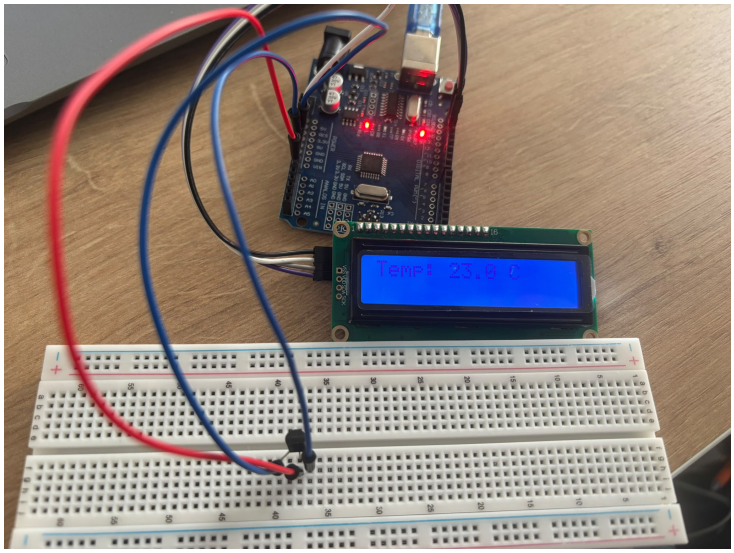


FIGURE 2.12 – Montage de l'affichage sur l'écran LCD

Chapitre 3

TP 3 : Afficheurs 7 segments, MATLAB/Simulink et Asservissement

3.1 Utilisation des afficheurs 7 segments

Un afficheur 7 segments permet de représenter des chiffres décimaux en allumant des segments spécifiques (constitués de LED internes). Pour cela, nous utiliserons la bibliothèque **SevSeg**, qui simplifie grandement l'affichage des chiffres sur l'afficheur sept segments en évitant d'avoir recours à de multiples instructions `digitalWrite()`.

a) Affichage du nombre 5 avec un afficheur simple Dans un premier temps, nous réalisons le câblage d'un seul afficheur 7 segments pour afficher le chiffre 5.

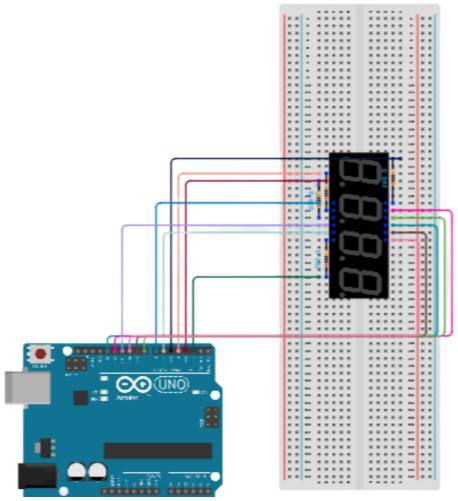


FIGURE 3.1 – Schéma du montage pour afficher le chiffre 5

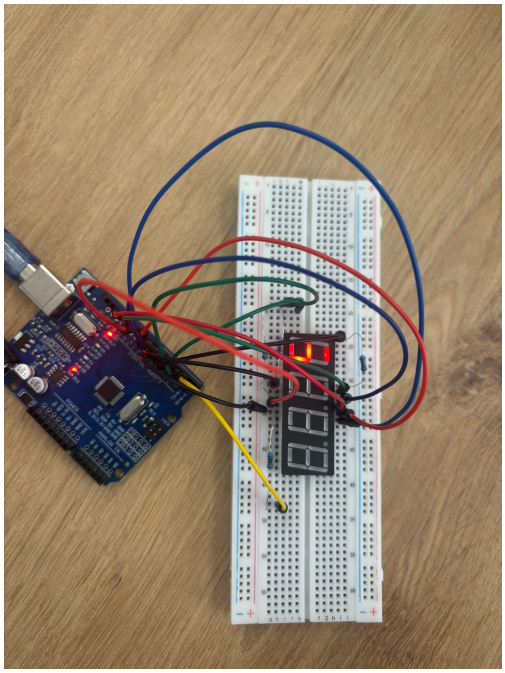


FIGURE 3.2 – Photo du montage de l'afficheur

```
#include "SevSeg.h"

SevSeg sevseg; // Instancier un objet controleur a sept segments

int cap = A0;  // Broche analogique connectee au potentiometre
int val;       // Variable pour lire la valeur de la broche analogique
```

```
7
8  void setup() {
9      byte numDigits = 4;
10     byte digitPins[] = { 2, 3, 4, 5 };
11     byte segmentPins[] = { 6, 7, 8, 9, 10, 11, 12, 13 };
12     bool resistorsOnSegments = false;      // 'false' : les
           resistances sont sur les broches de chiffres
13     byte hardwareConfig = COMMON_CATHODE;  // Voir README.md
           pour les options
14     bool updateWithDelays = false;         // Recommande par
           defaut
15     bool leadingZeros = false;             // 'true' pour
           afficher les zeros non significatifs
16     bool disableDecPoint = true;           // 'true' si le
           point decimal n'est pas connecte
17
18     sevseg.begin(hardwareConfig, numDigits, digitPins,
           segmentPins, resistorsOnSegments,
19     updateWithDelays, leadingZeros, disableDecPoint);
20     sevseg.setBrightness(100);
21 }
22
23 void loop() {
24     val = analogRead(cap);    // Lecture du potentiometre (0
           a 1023)
25     sevseg.setNumber(val, 0); // Affiche la valeur lue (
           remplace le 5 fixe)
26     sevseg.refreshDisplay();  // Doit tourner en continu sans
            delay()
27 }
```

Observations :L'afficheur s'allume correctement et forme le chiffre 5.

b) Affichage du nombre 25 et contrôle d'une LED en fonction de la température Nous combinons ici l'utilisation de deux afficheurs 7 segments pour afficher le nombre 25, tout en intégrant un capteur de température (comme le LM35 étudié au TP précédent). Une LED s'allume si la température dépasse 25 °C.

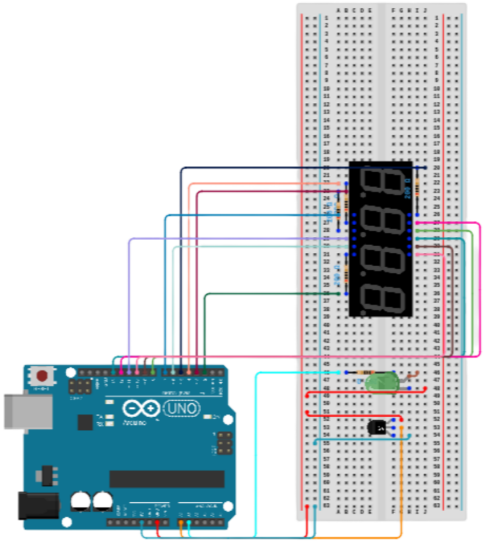


FIGURE 3.3 – Schéma du montage (Afficheurs + Capteur + LED)

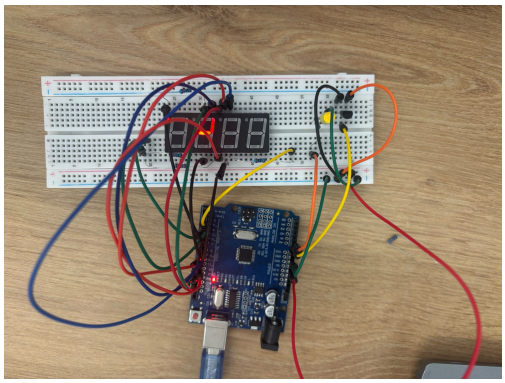


FIGURE 3.4 – Photo du montage (Afficheurs + Capteur + LED)

```
    void loop() {
        val = analogRead(cap);

        float voltage = val * (5.0 / 1023.0) ;
        float temperature = voltage * 100.0; // 10 mV par C
            donc T = tension * 100
        if (temperature > 15) {
            digitalWrite(15, HIGH);
        } else {
            digitalWrite(15, LOW);
        }
        sevseg.setNumber(temperature, 0);
        sevseg.refreshDisplay();  // Doit etre execute a
            plusieurs reprises
    }
```

Observations : l'afficheur sept segment montre bien la valeur perçu par le capteur de température et la LED s'allume quand l'afficher affiche une valeur supérieur a 15 comme indiqué dans notre programme.

c) Compteur de 1 à 99 avec deux afficheurs 7 segments Nous programmons ensuite la carte pour incrémenter et afficher automatiquement les nombres de 1 à 99 en utilisant nos deux afficheurs 7 segments. Nous allons juste changer le programme et pas la partie montage car elle est déjà fonctionnelle

```
1   void loop() {
2
3       for (int i =0 ; i<100 ; i++){ // notre boucle for
            pour afficher de 0 a 99
4           sevseg.setNumber(i, 0);
5
6           sevseg.refreshDisplay();
7           delay(1000);
8
9       };
10      delay(1000);
11  }
```

Observations : Nous observons bien le compteur fonctionner du chiffre 1 au nombre 99.

3.2 Interfaçage Arduino et MATLAB/Simulink

L'environnement Simulink permet de commander la carte Arduino en utilisant des blocs graphiques de la librairie "Simulink Support Package for Arduino Hardware".

d) Commande d'une LED depuis MATLAB/Simulink Nous réalisons un programme Matlab

FIGURE 3.5 – Modèle Simulink pour la commande de la LED

```
1   clear a;
2   a = arduino('/dev/ttyUSB0', 'Uno');
3
4   writeDigitalPin(a, 'D13', 1);
```

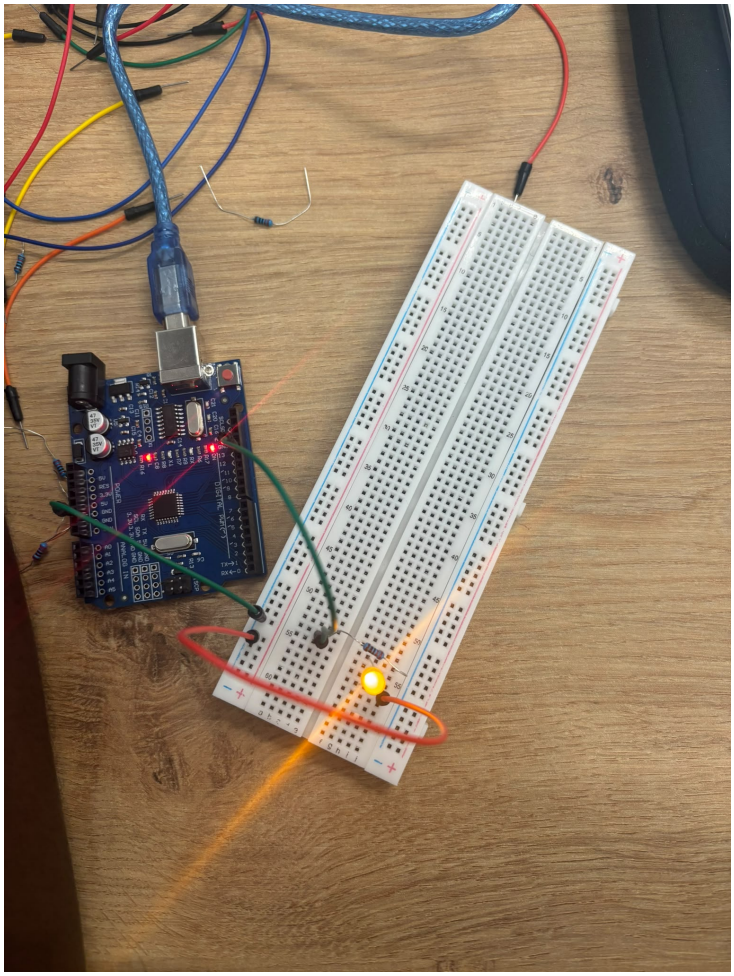


FIGURE 3.6 – Led contrôlé a l'aide de Matlab

Observations : La LED s'allume bien avec le code Matlab.

e) Commande d'un servomoteur avec MATLAB/Simulink De la même manière, nous contrôlons la position angulaire d'un servomoteur à d'un script Matlab.

```
clear a;
clear s;
a = arduino('/dev/ttyUSB0', 'Uno');
s = servo(a, 'D9');
angle = input('Entrez langle souhaite (entre 0 et 180) : ');
writePosition(s, angle / 180);
```

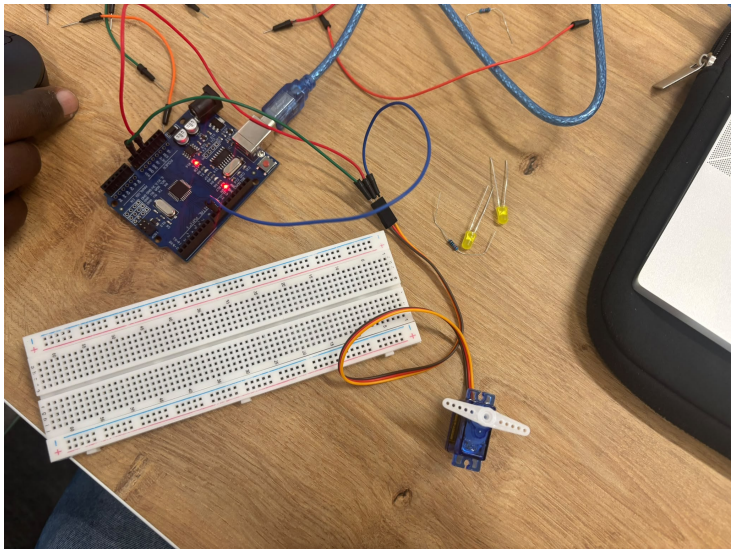


FIGURE 3.7 – Servomoteur contrôlé a l'aide de Matlab

Observations : Le moteur change de position en fonction de l'angle que nous lui donnons avec notre code.

3.3 Asservissement en vitesse d'un moteur avec régulateur PID

f) Composants et schéma pour l'asservissement Pour cette dernière partie, nous devons mettre en place un système complet d'asservissement en vitesse en utilisant un régulateur PID modélisé sous MATLAB/Simulink et implémenté sur Arduino.

Composants nécessaires :

- Carte Arduino Uno
- Moteur à courant continu (CC)
- Pont en H (Hacheur, ex : L298N) pour la partie puissance
- Capteur de Vitesse
- Alimentation externe pour le moteur

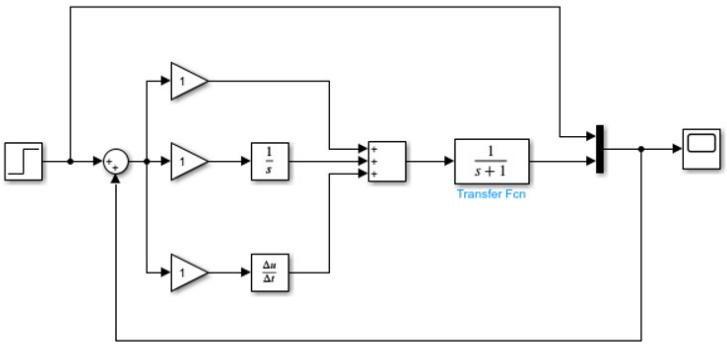


FIGURE 3.8 – Modèle Simulink du régulateur PID

Chapitre 4

TP : Commande des moteurs avec Arduino

4.1 Moteur à courant continu et module relais

Dans un système robotique, les moteurs (à courant continu, servomoteurs ou pas à pas) sont les actionneurs les plus courants pour générer du mouvement. Étant donné que les sorties de la carte Arduino ne délivrent pas assez de puissance (maximum 40 mA sous 5V), nous utilisons des modules relais. Ces derniers agissent comme des interrupteurs commandés, séparant le circuit de commande du circuit de puissance du moteur.

Exercice 1 : Commande d'un moteur dans les deux sens Dans cet exercice, nous utilisons deux relais pour créer un pont en H électromécanique. Cela permet de contrôler le sens de rotation du moteur à courant continu en toute sécurité, sans risque de court-circuit.

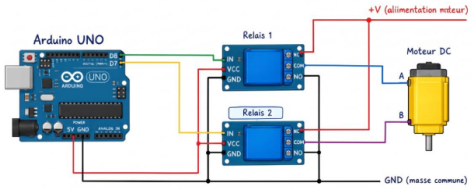


FIGURE 4.1 – Schéma de la commande avec deux relais

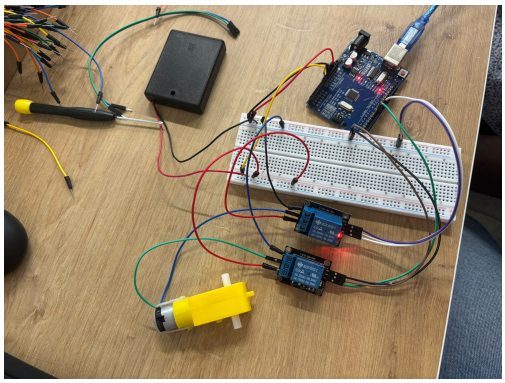


FIGURE 4.2 – Photo du montage avec relais et moteur DC

Réponses aux questions :

1. **Comportement du moteur :** Le moteur tourne dans un premier sens pendant 2 secondes, s'arrête 1 seconde, puis tourne dans le sens inverse pendant 2 secondes avant de s'arrêter à nouveau.
2. **État des relais :** En observant les LED, on constate que lorsque le Relais 1 est activé (LED allumée) et le Relais 2 au repos, le moteur tourne dans le sens 1. L'inverse produit la rotation dans le sens 2.
3. **Phase d'arrêt :** Cette phase est cruciale pour éviter un pic de courant trop important et pour éviter un court circuit.
4. **Modification du temps :** Pour modifier les durées, nous avons changé la valeur du premier `delay()` à `3000` et celle du second à `1000`.

Exercice 2 : Commande manuelle par bouton-poussoir Nous ajoutons un bouton-poussoir connecté à la broche D9. Le programme intègre une gestion de l'anti-rebond (`debounceDelay` de 50 ms) pour éviter de détecter plusieurs appuis lors d'une seule pression physique.

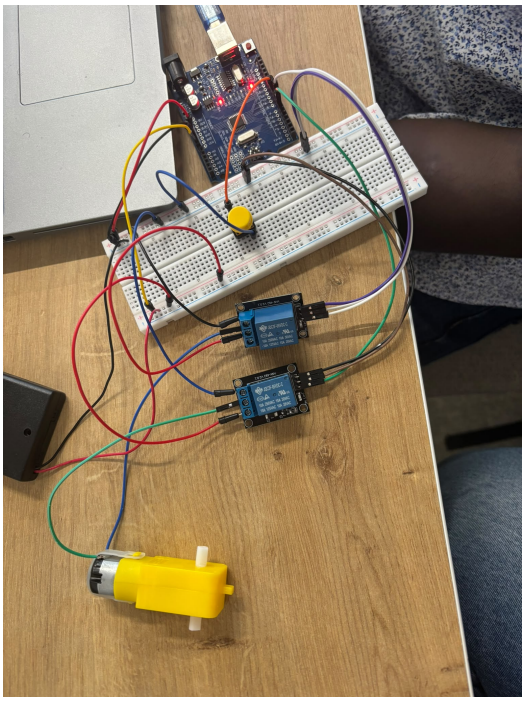


FIGURE 4.3 – Photo du montage avec 1 bouton-poussoir

Réponses aux questions :

1. **Utilisation de `INPUT_PULLUP` :** On utilise la résistance de tirage interne de l'Arduino (activée avec `INPUT_PULLUP`) pour simplifier le montage matériel. Cela évite d'avoir à câbler une résistance externe sur la plaque d'essai, tout en fixant un état logique clair (HIGH) lorsque le bouton n'est pas pressé.
2. **Effet de `debounceDelay` à 0 :** Si on met la variable `debounceDelay` à 0, l'anti-rebond est désactivé. Lors du test, on constate qu'un seul appui physique sur le bouton provoque des changements d'état aléatoires et multiples. Cela est dû aux rebonds mécaniques des contacts du bouton qui sont interprétés comme de multiples appuis successifs.
3. **Intégration de l'arrêt avant inversion :** La correction a été apportée dans les fonctions de mouvement. Passer d'abord par l'état d'arrêt avec un `delay(300)` avant d'inverser les relais permet d'éviter les pics

de courant, les contraintes mécaniques sur le moteur et l'usure prématurée des contacts des relais, en laissant le temps au moteur de ralentir.

4. **Ajout d'un second bouton :** Pour utiliser deux boutons (un pour marche/arrêt et un pour le sens de rotation), il faudrait déclarer une seconde entrée avec `INPUT_PULLUP` (par exemple sur D10). Le premier bouton basculerait l'état d'une variable booléenne `moteurActif`, et le second bouton modifierait l'état d'une variable de sens. Les fonctions de commande des relais (`moveForward` et `moveReverse`) ne seraient appelées que si `moteurActif` est vrai.

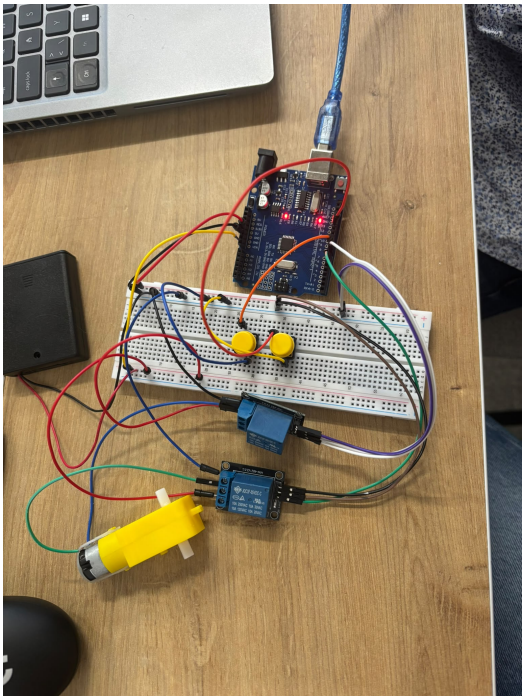


FIGURE 4.4 – Montage avec deux boutons poussoir

4.2 Signal PWM et servomoteur

La technique PWM (Pulse Width Modulation) permet de simuler une tension analogique en faisant varier le rapport cyclique d'un signal carré. Cependant, contrairement à un moteur DC classique, un servomoteur nécessite un signal PWM particulier : des impulsions précises de 1 à 2 ms répétées toutes les 20 ms (50 Hz) pour définir un angle cible.

Exercice 3 : Positionner un servomoteur à des angles précis Nous utilisons la bibliothèque `Servo.h` au lieu de la commande `analogWrite()`, car cette dernière ne fonctionne pas à la bonne fréquence. Cette bibliothèque permet d'envoyer facilement un angle de consigne (entre 0 et 180 degrés) au servomoteur SG90.

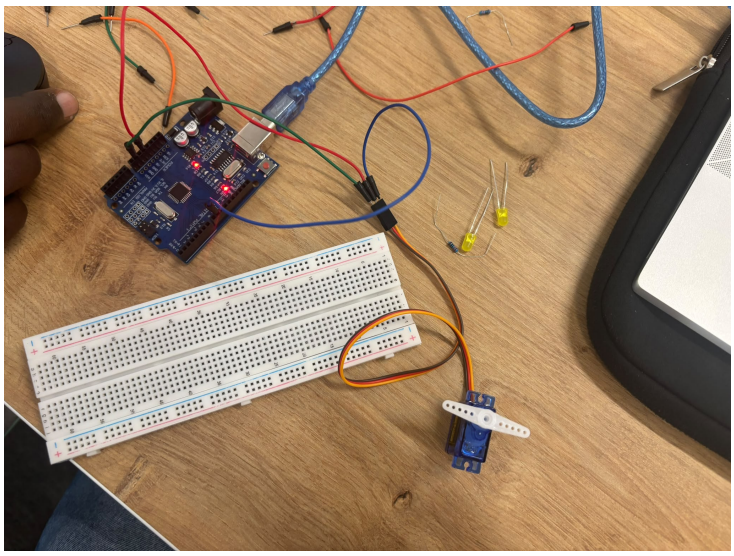


FIGURE 4.5 – Servomoteur contrôlé a l'aide de Matlab

Réponses aux questions :

1. **Différence entre 3-a et 3-b :** Dans le programme 3-a, le mouvement du servomoteur est très saccadé et brutal : il saute directement d'un angle à l'autre (0°, 90°, 180°, 45°) à sa vitesse maximale. Dans le programme 3-b, le mouvement est fluide et progressif (balayage). Le programme décompose le trajet en petits pas de 1° avec un délai de 15 ms entre chaque pas.
2. **Durée d'un aller (0° → 180°) :** Dans le programme 3-b, le balayage s'effectue sur 180 pas (de 0 à 180). Comme le programme attend 15 ms à chaque pas (`delay(15)`), la durée totale de l'aller est d'environ : $180 \times 15 = 2700$ ms, soit 2,7 secondes.
3. **Modification du balayage (aller rapide, retour lent) :** Pour obtenir cet effet, nous avons modifié la première boucle `for` (aller) en réduisant le délai à `delay(5)` pour un mouvement rapide, et modifié la seconde boucle `for` (retour) en augmentant le délai à `delay(30)` pour un mouvement plus lent.

```
1     for (int angle = 180; angle >= 0; angle -= 1) {
```

```
        monServo.write(angle);
        delay(5); // ici on change la duree pour changer
            la vitesse
    }
    delay(1000); // Pause d'une seconde avant de
        recommencer
    }
```

4. **Précision des butées (0° et 180°) :** Par défaut, les servomoteurs bon marché comme le SG90 n'atteignent pas toujours exactement 0° et 180° avec les valeurs d'impulsions standards (1000 µs à 2000 µs). En utilisant `monServo.attach(9, 500, 2400)`, on redéfinit les bornes temporelles du signal PWM (500 µs pour 0° et 2400 µs pour 180°). Avec ce réglage, on constate que le servomoteur exploite une plus grande amplitude mécanique et se rapproche beaucoup plus des vraies butées 0° et 180°.

Exercice 4 : Commande du servomoteur par potentiomètre Pour piloter manuellement l'angle du servomoteur, nous lisons la tension aux bornes d'un potentiomètre sur l'entrée analogique A0. La fonction `map()` nous permet ensuite de convertir cette lecture (de 0 à 1023) en un angle proportionnel compris entre 0° et 180°.

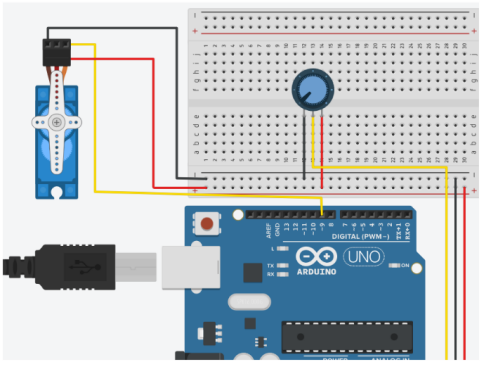


FIGURE 4.6 – Schéma de la commande par potentiomètre

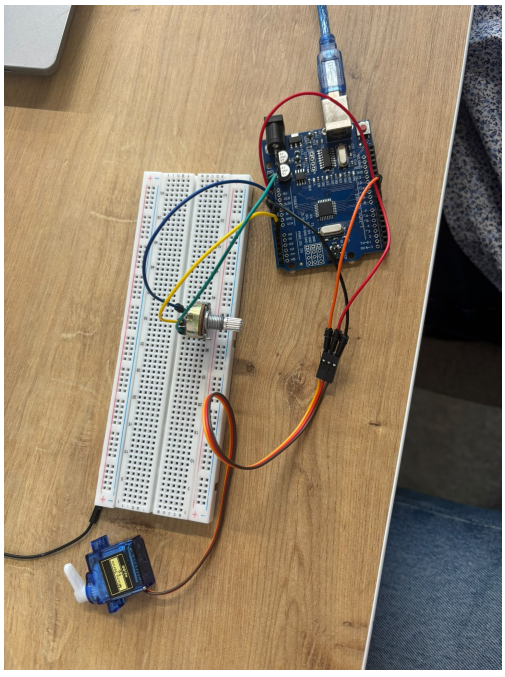


FIGURE 4.7 – Photo du montage avec potentiomètre et servomoteur

Réponses aux questions :

1. **Relevés du moniteur série :**
   - Position 1 : Potentiomètre = 0 → Angle = 0°
   - Position 2 : Potentiomètre = 512 → Angle = 90°
   - Position 3 : Potentiomètre = 1023 → Angle = 180°
2. **Calcul pour la valeur 512 :** La fonction `map()` réalise un produit en croix (ou interpolation linéaire). La plage du potentiomètre va de 0 à 1023 (amplitude de 1023), et la plage de l'angle va de 0 à 180 (amplitude de 180). Le calcul est : Angle = $\frac{512 \times 180}{1023} \approx 90,08°$. La valeur affichée est donc bien 90°, ce qui correspond à la position centrale.
3. **Limitation du mouvement (30° à 150°) :** Pour limiter la plage de mouvement du servomoteur, il suffit de modifier les arguments de sortie de la fonction `map()` dans le programme. La ligne devient : `int angle = map(valeurPot, 0, 1023, 30, 150);`. Ainsi, même si le potentiomètre est à 0, le servo n'ira pas en dessous de 30°, et s'il est à 1023, il n'ira pas au-delà de 150°.
4. **Solution logicielle contre les tremblements :** Pour résoudre cela on enregistre l'ancien angle, et on n'envoie la commande `monServo.write(angle)` que si la différence absolue entre le nouvel angle calculé et l'ancien est supérieure ou égale à 2°.

```
      // 3. Commande du servomoteur avec correction
         logiciel de l'angle
    int pastAngle = 0 ;

    if (abs(newAngle - pastAngle) > 2 ) {
        monServo.write(newAngle);
        pastAngle = newAngle;
    }
    else{
        monServo.write(pastAngle);
    }
```